

CO-3 AT-2

17. MUSIC Algorithm Simulation

Simulate the MUSIC algorithm to resolve two closely spaced sinusoidal frequencies in the presence of noise.

Given Data:

- Frequencies = 500 Hz and 520 Hz
- Sampling Frequency = 5 kHz
- SNR = 10 dB

Tasks:

1. Generate the signal.
2. Form covariance matrix.
3. Perform eigenvalue decomposition.
4. Implement MUSIC algorithm.
5. Estimate frequencies.
6. Compare with periodogram results.

INTRODUCTION

The Multiple Signal Classification (MUSIC) algorithm is a high-resolution spectral estimation technique used to identify closely spaced frequency components in noisy signals. It works by separating the signal subspace from the noise subspace using eigenvalue decomposition of the covariance matrix. Compared to conventional methods like the periodogram, MUSIC provides much better frequency resolution. It is widely used in radar, sonar, wireless communication, and direction-of-arrival (DOA) estimation.

OBJECTIVE

To implement the MUSIC algorithm in MATLAB for estimating two closely spaced sinusoidal frequencies (500 Hz and 520 Hz) in the presence of 10 dB noise. To compare the frequency estimation performance of the MUSIC algorithm with the conventional periodogram method.

TOOLS USED

MATLAB

PROGRAM

```
clc;
```

```
clear;
```

```
close all;
```

```
%% Step 1: Generate the Signal
```

```
Fs = 5000;      % Sampling Frequency (Hz)
```

```
f1 = 500;      % Frequency 1
```

```
f2 = 520;      % Frequency 2
```

```
N = 1024;      % Number of samples
SNR = 10;      % Signal-to-Noise Ratio (dB)
```

```
t = (0:N-1)/Fs;
```

```
% Two sinusoidal signals
x = sin(2*pi*f1*t) + sin(2*pi*f2*t);
```

```
% Add White Gaussian Noise
x = awgn(x,SNR,'measured');
```

```
%% Step 2: Form Covariance Matrix
```

```
M = 20;        % Number of array elements (subspace dimension)
```

```
X = zeros(M,N-M+1);
```

```
for i = 1:N-M+1
    X(:,i) = x(i:i+M-1)';
end
```

```
% Covariance Matrix
R = (X*X')/(N-M+1);
```

```
%% Step 3: Eigenvalue Decomposition
```

```
[E,D] = eig(R);
```

```
% Sort Eigenvalues in descending order
```

```
[eigenvalues,index] = sort(diag(D),'descend');
```

```
E = E(:,index);
```

```
% Number of signal sources
```

```
P = 2;
```

```
% Noise Subspace
```

```
En = E(:,P+1:end);
```

```
%% Step 4: MUSIC Algorithm
```

```
f = 0:0.1:1000;      % Frequency search range
```

```
Pmusic = zeros(length(f),1);
```

```
for k = 1:length(f)
```

```
    a = exp(-1j*2*pi*(0:M-1)*f(k)/Fs);
```

```
    Pmusic(k) = 1/(a'*En*En'*a);
```

```
end
```

```
Pmusic = abs(Pmusic);
```

```
Pmusic = Pmusic/max(Pmusic);
```

```
%% Step 5: Estimate Frequencies
```

```
[pks,locs] = findpeaks(Pmusic,f,'SortStr','descend');
```

```
Estimated_Frequencies = sort(locs(1:2));
```

```
disp('Estimated Frequencies using MUSIC (Hz)');
```

```
disp(Estimated_Frequencies);
```

```
%% Step 6: Periodogram
```

```
[pxx,freq] = periodogram(x,[],4096,Fs);
```

```
%% Plot Results
```

```
figure;
```

```
subplot(2,1,1)
```

```
plot(freq,10*log10(pxx),'LineWidth',1.5)
```

```
grid on
```

```
xlabel('Frequency (Hz)')
```

```
ylabel('Power (dB)')
```

```
title('Periodogram')
```

```
subplot(2,1,2)
```

```
plot(f,10*log10(Pmusic),'r','LineWidth',1.5)
```

```
grid on
```

```
xlabel('Frequency (Hz)')
```

```
ylabel('Pseudo Spectrum (dB)')
```

```
title('MUSIC Spectrum')
```

OUTPUT

Estimated Frequencies using MUSIC (Hz)

140.2000 509.6000

